DAY19-COOKIES-SESSIONS

THEORY (answer all 20 questions in your own words):

1. Why does a web application need state-management mechanisms if HTTP is stateless?
    - HTTP is a stateless protocol.
    - Server does not automatically remember previous requests.
    - Cookies and sessions maintain information such as login status and shopping carts.

2. What is a cookie?
    - A small piece of data stored in user's web browser.
    - Stores preferences and identifiers.
    - It helps maintain user information between requests.

3. What is Set-Cookie?
    - It is a HTTP response header.
    - It is sent from server to the browser.
    - It instructs browser to store cookie.

4. What is the Cookie request header?
    - It is an HTTP request header.
    - Sent from browser to server.
    - Contains cookies previously stored by browser.

5. Explain the basic cookie request/response flow.
    - Browser send request to website.
    - Server respond with Set-cookie header
    - Contain cookie previously stored by browser.
    - Server uses cookie data to identify the server.

6. What is a web session?
    - A web session represents a user's interaction with a website.
    - It can store information such as login status, server commonly maintaing session data.

7. What is a session ID?
    - A session ID is a unique identifier assigned to a user's session.
    - It helps the server recognise the user's session, commonly stored in browser's cookie.

DAY19-COOKIES-SESSIONS

8. Explain the basic login-session flow.

    - The user enters their username and password.
    - The server verifies the login credentials.
    - The server creates a new session.
    - The server sends a session ID to the browser, commonly through a cookie.
    - The browser sends the session ID with future requests to maintain the login.

9. Why should sessions expire?
    - This reduces the risk of unauthorised access.
    - It protects accounts on shared or public computers.
    - It limits the time an attacker can misuse a stolen session ID.

10. What happens conceptually during logout?

    - The user clicks the logout button.
    - The browser sends a logout request to the server.
    - The server invalidates or destroys the user's session.
    - The session cookie may also be cleared from the browser.
    - The user must log in again to access protected resources.

11. What is the Secure cookie attribute?
    - Secure is a cookie security attribute.
    - It restricts the cookie to secure HTTP connections, help protect cookie during transmissions.

12. What is HttpOnly?
    - It is a cookie security attribute.
    - It prevents normal Java Script APIs from accessing the cookie.
    - It helps cookie from XSS attack.

13. What is SameSite?
    - It is also a cookie security attribute
    - It controls cookie sending behaviour in cross-site requests.
    - It helps protect against Cross-Site Request Forgery (CSRF).

14. Explain Strict, Lax and None at a beginner level.
    - Strict: Cookies are sent only in same-site contexts.
    - Lax: Cookies are generally sent in same-site requests and certain top-level cross-site navigations.
    - None: Allows cookies in cross-site contexts.

DAY19-COOKIES-SESSIONS

15. What do Expires and Max-Age control?
    - Expires specifies the date and time when a cookie expires.
    - Max-Age specifies how many seconds a cookie remains valid.

16. What is Path used for?
    - It specifies which URL paths can receive the cookie.
    - It controls where the browser sends the cookie within a website.

17. What is localStorage?
    - localStorage is a browser-side storage mechanism, stores data as a key-value pairs.
    - Data remains stored even after the browser is closed.

18. What is sessionStorage?
    - sessionStorage is a browser-side storage mechanism, stores data as a key-value pairs.
    - Data is available within a browser tab's page session.

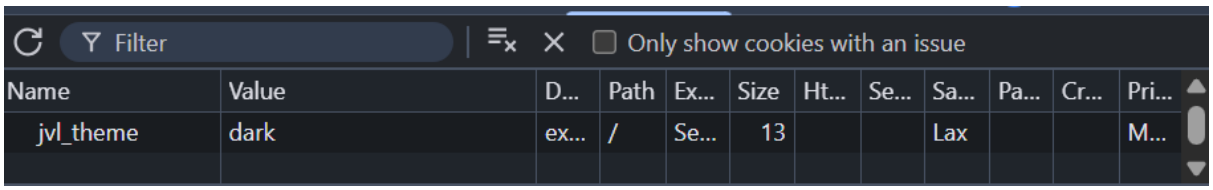
19. Why is sessionStorage different from a server-side login session?
    - sessionStorage stores data in the user's browser and a server-side login session stores session information on the server.
    - sessionStorage is directly accessible through JavaScript and server-side session is commonly identified using a session ID.

20. Why should real session cookies never be shared in assignments or screenshots?
    - Session cookies may contain authentication identifiers.
    - Attackers may impersonate the legitimate user without knowing their password.
    - Always hide or redact real cookies and use dummy values in assignments.

DAY 19 PRACTICAL LAB — Cookie & Session Inspector Lab (SAFE, fictional values only)

THIS IS A FAKE TRAINING LAB. NO REAL LOGIN SESSION IS USED. Do NOT inspect real login cookies from personal accounts.

TASK 1 — Identify the cookie: shown "Set-Cookie: theme=dark" → Cookie Name: theme, Cookie Value: dark.



DAY19-COOKIES-SESSIONS

PRACTICAL PART (fictional example only — do NOT screenshot real authentication cookies):

Document this training cookie:

Set-Cookie: training_session=JVL123; Secure; HttpOnly; SameSite=Lax; Max-Age=3600; Path=/

Record: Cookie Name · Cookie Value · Secure Status · HttpOnly Status · SameSite Value · Max-Age · Path.

Then write 3-5 sentences explaining why these attributes matter. let's do this

- Cookie Name: training_session
- Cookie Value: JVL123
- Secure Status: HttpOnly
- HTTP only status: Enabled - Yes
- SameSite value: Lax
- Max-age: 3600 seconds
- Path: / root path

Cookie Attribute analysis:

Secure ensures the cookie is only sent over HTTPS (i.e. during transmission it is protected). HttpOnly prevents the cookie from being accessed via normal JavaScript APIs which in turn reduces the risk of it being stolen via an XSS attack. SameSite=Lax controls the transmission of the cookie across sites and helps prevent CSRF attacks. Max-Age=3600 means the cookie is valid for one hour and Path=/ means it will be sent to any path that matches. All of these attributes improve the security of the cookie, though they don't in any way guarantee that the application as a whole is secure.